# From String to Comparable

## Interfaces, a comparable Counter, and generics

Marcus Birkenkrahe

COSC 2230 · CS III · Augustana University

Fall 2026

# By the end of this session you can

1. explain what a Java interface is: a promise to provide methods
2. turn the `String` insertion sort into one for any `Comparable` type
3. make your own class comparable with `implements Comparable<T>`
4. avoid the subtraction trap in `compareTo()`
5. read the "unchecked" warning and remove it with a generic method

# The problem: one sort per type?

`Insertion.sort(String[] a)` sorts strings only.

**How would you sort an `Integer[]`, a `Double[]`, a `Counter[]`?**

**Copy and replace**

One copy per type – same algorithm, every bug fixed in every copy.

**What the algorithm really needs**

The algorithm needs only one thing from the data: that two items of any data type can be compared.

# Interfaces: a promise

A Java **interface** lists methods without bodies. A class that **implements** it promises to provide them – the compiler checks.

```
public interface Comparable<Key>
{
    int compareTo(Key b);
}
```

- `String`, `Integer`, `Double` already implement it
- that is why `"eve".compareTo("oscar")` worked in binary search
- the order it defines: the type's **natural order**
- `<Key>`: both objects must have the same type (end of today)

Defined in `java.lang.Comparable`, part of the JDK – no import needed

# InsertionComparable

→ **Switch to editor · code-along 1**

1. Copy **`Insertion.java`** to **`InsertionComparable.java`**
   code-along: **`birkenkrahe.github.io/go/insertion-sort`**
2. Replace **`String`** by **`Comparable`** in `sort()` and `exchange()` – nothing else changes
3. Test client: sort a `String[]`, an `Integer[]` and a `Double[]`
   copy it from **`birkenkrahe.github.io/go/InsertionComparable-main`**
4. Compile – and read what the compiler says

classpath: `.:stdlib.jar`

# One method, three types

```
public static void sort(Comparable[] a)
{
    int N = a.length;
    for (int i = 1; i < N; i++)
        for (int j = i; j > 0; j--)
            if (a[j-1].compareTo(a[j]) > 0)
                exchange(a, j-1, j);
            else break;
}
```

```
[and, but, had, him, ...]
[3, 7, 19, 25, 42, 88]
[0.58, 1.41, 2.71, 3.14]
```

`int` is not an object: `Integer` is its **wrapper class**. Java converts automatically (**autoboxing**), so initializing with `{...}` works.

# The warning: a raw type

```
$ javac -Xlint:unchecked -cp .:stdlib.jar InsertionComparable.java
warning: [unchecked] unchecked call to compareTo(T) as a member
         of the raw type Comparable
```

`Comparable` without <. . .> is a **raw type**: the compiler cannot check that both objects have the same type.

## What could go wrong?

```
Comparable[] a = { "eve", 42, "bob" };
InsertionComparable.sort(a);
```

```
ClassCastException: class
java.lang.Integer cannot be cast
to class java.lang.String
```

It compiles – and crashes at run time.

# What compareTo() must guarantee

`a.compareTo(b)`: **negative**, **0** or **positive** – test the sign.

## A total order

- **antisymmetric:** if $a \leq b$ and $b \leq a$, then $a = b$
- **transitive:** if $a \leq b$ and $b \leq c$, then $a \leq c$
- **total:** $a \leq b$ or $b \leq a$ – any two items can be compared

Otherwise the results are unpredictable: insertion sort trusts every compare.

```
error: incompatible types: Plain[] cannot be converted to Comparable[]
```

A class without the promise is rejected at compile time.

# Your Counter from week 1, comparable

→ **Switch to editor · code-along 2**

1. Open **your `Counter.java`** from week 1
   no file? **birkenkrahe.github.io/go/Counter-java**
2. Add **`implements Comparable<Counter>`** to the class declaration
3. Write **`compareTo(Counter b)`**: compare the counts
4. Add **`toString()`** to print a counter as its count
5. Test client: N counters with random counts, sort them with `InsertionComparable`
   copy it from **birkenkrahe.github.io/go/Counter-main**

files: `Counter.java` · `InsertionComparable.java`

# A comparable Counter

```
public class Counter
             implements Comparable<Counter>
{
    private int count;
    ...
    public int compareTo(Counter b)
    {
        if      (count < b.count) return -1;
        else if (count > b.count) return +1;
        else                      return  0;
    }
    public String toString()
    { return "" + count; }
}
```

```
$ java Counter 10
[90, 80, 91, 74, 23, ...]
[23, 36, 43, 54, 74, ...]
```

**Why not <= and >=?**

→ 3 answers needed; <= is the caller's.

**b.count is private – why allowed?**

→ Private to the class, not to the object.

**Why does the old sort work?**

→ It relies only on the promise.

# Review: `Comparable`, `compareTo()`, autoboxing

1. **What is `Comparable`, and which classes implement it?**
   → An interface with one method, `compareTo()`. `String`, `Integer`, `Double` implement it; your own classes can.
2. **What must `compareTo()` return, and what must it guarantee?**
   → Negative, 0 or positive. A total order: antisymmetric, transitive, total – the compiler does not check it.
3. **What is autoboxing, and what is it for?**
   → `int` → `Integer` automatically, so primitive values can go where objects are needed: `Integer[]` into `sort(Comparable[] a)`.

# Pitfall: compare by subtraction

A shortcut you will see online:

```
public int compareTo(Counter b) { return count - b.count; }
```

The sign is right ... until the subtraction overflows:

```
a = 2,000,000,000   b = -2,000,000,000
a - b                 = -294967296
Integer.compare(a, b) = 1
```

**a is larger, but a − b is negative**

The same trap as `(lo + hi) / 2` in binary search.

**Use the `if` chain or `Integer.compare()`**

```
public int compareTo(Counter b)
{ return Integer.compare(count, b.count); }
```

→ `CompareOverflow.java` (demo only)

# InsertionGeneric: remove the warning

→ **Switch to editor · code-along 3**

1. Copy **InsertionComparable.java** to **InsertionGeneric.java** and rename the class
   no file? **birkenkrahe.github.io/go/insertion-comparable**
2. `sort(Comparable[] a)` becomes
   **`<Key extends Comparable<Key>> void sort(Key[] a)`**
3. `exchange(Comparable[] a, ...)` becomes **`<Key> void exchange(Key[] a, ...)`**,
   with `Key t` instead of `Comparable t`
4. Compile with **`javac -Xlint:unchecked`**: is the warning gone?

files: `InsertionComparable.java` → `InsertionGeneric.java`

# Generics: the warning goes away

```
public static <Key extends Comparable<Key>> void sort(Key[] a)
```

`Key`: any type that is comparable with itself. No warning – and the mixed array is now caught by the compiler:

```
error: method sort in class InsertionGeneric cannot be applied
  required: Key[]   found: Object[]
```

**Raw type**

error at run time – maybe in front of a user

**Generic**

error at compile time – before the program runs

Next in §4.3: `Stack<Item>`, `Queue<Item>`, `ST<Key, Value>` – write the code once, let the compiler check the types.

# Practice: a named Counter

**PRACTICE · ON YOUR OWN · 10 MIN**

1. Get `votes.txt`, `main()` (and `InsertionGeneric.java`): **`birkenkrahe.github.io/go/Counter-votes`**
2. In your `Counter.java` (none? `go/Counter-comparable`): field `private final String name;`, constructor `Counter(String name, int count)`
3. `compareTo()`: `Integer.compare()` on the counts; equal counts: `compareTo()` on the names
4. `toString()`: name and count, e.g. `carol 60`
5. Replace `main()` with the given test client
6. Compile and run until the output matches

Expected output:

```
carol 60
dave 72
wendy 72
alice 85
carlos 85
walter 90
```

```
javac -cp .:stdlib.jar InsertionGeneric.java Counter.java
java -cp .:stdlib.jar Counter < votes.txt
```

Done early? Why do dave and wendy need the name – but alice and carlos don't?

# Practice: what changes in Counter

```
private final String name;
private int count;

public Counter(String name, int count)
{ this.name = name; this.count = count; }

public int compareTo(Counter b)
{
  if (count != b.count)
    return Integer.compare(count, b.count);
  return name.compareTo(b.name);
}

public String toString()
{ return name + " " + count; }
```

- `final`: every constructor must set `name` – delete `Counter(int)`
- `this.`: the parameters hide the fields
- counts first, names only on a tie: a total order
- `" "`, not `""`: `carol 60`, not `carol60`
- new `main()`: the old one calls `new Counter(0)`
- compile error? `java` runs the old `Counter.class`

# Wrap-up: from String to Comparable

- An interface is a promise: `implements Comparable<T>` promises `compareTo()`
- One insertion sort for every comparable type
- `compareTo()`: negative, 0, positive; a total order; no subtraction
- Raw type: warning and run-time errors – generic: errors at compile time

**Next: mergesort – the N log N sort.**

# Checkout question

**You have 1,000 exam papers to sort by name, and two friends to help. How do you split the work?**